```
-- ========================================
-- ASSIGNMENT 2: DDL & Integrity Constraints
-- ========================================

-- Department Table Creation
CREATE TABLE Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50) UNIQUE NOT NULL
);

-- Student Table Creation
CREATE TABLE Student (
    Stud_ID INT PRIMARY KEY,
    Stud_Name VARCHAR(50) NOT NULL,
    Age INT CHECK (Age >= 18),
    Email VARCHAR(100) UNIQUE,
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);

-- Inserting Data
INSERT INTO Department VALUES
    (1, 'Computer Engineering'),
    (2, 'Information Technology'),
    (3, 'Electronics');

INSERT INTO Student VALUES
    (101, 'Amit Sharma', 20, 'amit@gmail.com', 1),
    (102, 'Neha Patil', 19, 'neha@gmail.com', 2),
    (103, 'Rahul Verma', 21, 'rahul@gmail.com', 1);

-- Basic Selections
SELECT * FROM Department;
SELECT * FROM Student;

-- Altering Table
ALTER TABLE Student ADD City VARCHAR(30);
desc Student;

-- Renaming Table
RENAME TABLE Student TO Student_Details;
show tables;
select * from student_details;

-- Truncate and Drop
TRUNCATE TABLE Student_Details;
select * from student_details;
desc Student_details;
DROP TABLE Student_Details;

-- ----------------------------------------
-- Integrity Constraints Application
-- ----------------------------------------
CREATE TABLE Course (
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(50) NOT NULL UNIQUE,
    Duration INT CHECK (Duration >= 1)
);
show tables;

CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
```

```
    Age INT CHECK (Age>=18),
    Course_ID INT,
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);
show tables;

-- Valid Inserts for testing constraints
INSERT INTO Course VALUES (1, 'Computer Science', 4), (5, 'Data Science', 2);
INSERT INTO Student VALUES (101, 'Amit', 'amit@gmail.com', 20, 1);

-- Constraint Violations (These will intentionally throw errors to prove
constraints work)
INSERT INTO Course VALUES (3, NULL, 5);                                    -
- Fails: NOT NULL constraint
INSERT INTO Student VALUES (103, 'Ravi', 'amit@gmail.com', 21, 1);         -
- Fails: UNIQUE constraint (amit@gmail.com exists)
INSERT INTO Student VALUES (104, 'Pooja', 'pooja@gmail.com', 16, 1);       -
- Fails: CHECK constraint (Age < 18)
INSERT INTO Student VALUES (105, 'Kiran', 'kiran@gmail.com', 20, 99);      -
- Fails: FOREIGN KEY constraint (Course 99 doesn't exist)


-- ========================================
-- ASSIGNMENT 3: DML, DCL, and TCL
-- ========================================

-- DML Tables
CREATE TABLE Book (
    Book_ID INT PRIMARY KEY,
    Book_Name VARCHAR(50),
    Author VARCHAR(50),
    Price INT,
    Category VARCHAR(30)
);

CREATE TABLE Issued_Book (
    Book_Name VARCHAR(50)
);

INSERT INTO Book VALUES
    (1, 'Database Systems', 'Navathe', 500, 'Education'),
    (2, 'Operating System', 'Silberschatz', 650, 'Education'),
    (3, 'C Programming', 'Dennis Ritchie', 300, 'Programming'),
    (4, 'Python Basics', 'Guido', 400, 'Programming');

show tables;
select * from Book;

-- Update and Delete
UPDATE Book SET Price = Price + (Price * 0.10) WHERE Category =
'Programming';
select * from Book;

DELETE FROM Book WHERE Price < 350;
select * from Book;

-- Arithmetic, Logical, and Pattern Matching
SELECT Book_Name, Price, Price + 50 AS New_Price FROM Book;
SELECT * FROM Book WHERE Category = 'Education' AND Price > 500;
SELECT * FROM Book WHERE Book_Name LIKE 'D%';

-- String Functions
SELECT UPPER(Book_Name) AS Book_Upper, LOWER(Author) AS Author_Lower FROM
Book;
```

```
SELECT Book_Name, LENGTH(Book_Name) AS Name_Length FROM Book;
SELECT SUBSTRING(Book_Name, 1, 4) AS Short_Name FROM Book;

-- Set Operators
INSERT INTO Issued_Book VALUES ('Database Systems'), ('Python Basics');
select * from Issued_Book;

SELECT Book_Name FROM Book UNION SELECT Book_Name FROM Issued_Book;
SELECT Book_Name FROM Book INTERSECT SELECT Book_Name FROM Issued_Book;
SELECT Book_Name FROM Book EXCEPT SELECT Book_Name FROM Issued_Book;

-- DCL (Data Control Language)
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE student_dcl (
    roll_no INT PRIMARY KEY,
    name VARCHAR(50),
    marks INT
);

INSERT INTO student_dcl VALUES (1,'Amit',85), (2, 'Neha',90), (3,'Ravi', 78);

CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'admin123';
CREATE USER 'faculty1'@'localhost' IDENTIFIED BY 'faculty123';
CREATE USER 'student1'@'localhost' IDENTIFIED BY 'student123';

CREATE ROLE 'admin_role';
CREATE ROLE 'faculty_role';
CREATE ROLE 'student_role';

GRANT ALL PRIVILEGES ON college_db.* TO 'admin_role';
GRANT SELECT, UPDATE ON college_db.student_dcl TO 'faculty_role';
GRANT SELECT ON college_db.student_dcl TO 'student_role';

GRANT 'admin_role' TO 'admin1'@'localhost';
GRANT 'faculty_role' TO 'faculty1'@'localhost';
GRANT 'student_role' TO 'student1'@'localhost';

SET DEFAULT ROLE ALL TO 'admin1'@'localhost', 'faculty1'@'localhost',
'student1'@'localhost';

SHOW GRANTS FOR 'faculty1'@'localhost';

-- Testing Access (assuming logged in as respective user)
SELECT * FROM college_db.student_dcl;
-- INSERT INTO college_db.student_dcl VALUES (4, 'Pooja',88); -- Fails for
student

REVOKE UPDATE ON college_db.student_dcl FROM 'faculty_role';
SHOW GRANTS FOR 'faculty1'@'localhost';

DROP ROLE 'student_role';
DROP USER 'student1'@'localhost';

-- TCL (Transaction Control Language)
CREATE TABLE Account (
    Acc_No INT PRIMARY KEY,
    Acc_Holder VARCHAR(50),
    Balance INT
);

INSERT INTO Account VALUES (101, 'Amit Patil', 10000), (102, 'Neha Sharma',
15000);
```

```
select * from Account;

START TRANSACTION;
UPDATE Account SET Balance = Balance - 2000 WHERE Acc_No = 101;
UPDATE Account SET Balance = Balance + 2000 WHERE Acc_No = 102;
SAVEPOINT transfer_done;

select * from Account;

UPDATE Account SET Balance = Balance - 5000 WHERE Acc_No = 101;
ROLLBACK TO transfer_done;
select * from Account;

COMMIT;
select * from Account;


-- ========================================
-- ASSIGNMENT 5: Aggregate Functions & Grouping
-- ========================================

CREATE TABLE Student_Result (
    Student_ID INT,
    Student_Name VARCHAR(50),
    Subject VARCHAR(30),
    Marks INT,
    Department VARCHAR(30)
);

INSERT INTO Student_Result VALUES
    (1, 'Amit', 'DBMS', 78, 'Computer'),
    (2, 'Neha', 'DBMS', 85, 'Computer'),
    (3, 'Rahul', 'OS', 72, 'Computer'),
    (4, 'Sneha', 'OS', 88, 'IT'),
    (5, 'Pooja', 'DBMS', 91, 'IT'),
    (6, 'Ravi', 'OS', 65, 'IT');

SELECT Department, COUNT(Student_ID) AS Total_Students FROM Student_Result
GROUP BY Department;
SELECT Subject, AVG(Marks) AS Avg_Marks FROM Student_Result GROUP BY Subject;
SELECT Department, MAX(Marks) AS Highest_Marks, MIN(Marks) AS Lowest_Marks
FROM Student_Result GROUP BY Department;
SELECT Department, SUM(Marks) AS Total_Marks FROM Student_Result GROUP BY
Department;
SELECT Department, AVG(Marks) AS Avg_Marks FROM Student_Result GROUP BY
Department HAVING AVG(Marks) > 80;
SELECT Subject, COUNT(Student_ID) AS Student_Count FROM Student_Result GROUP
BY Subject HAVING COUNT(Student_ID) > 2;
SELECT Department, AVG(Marks) AS Avg_Marks FROM Student_Result WHERE Marks >=
70 GROUP BY Department HAVING AVG(Marks) > 75;


-- ========================================
-- ASSIGNMENT 5: JOIN Operations and Views
-- ========================================

CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    city VARCHAR(30)
);

CREATE TABLE Product (
    product_id INT PRIMARY KEY,
```

```
    product_name VARCHAR(50),
    price DECIMAL(10,2)
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    quantity INT,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);

INSERT INTO Customer VALUES (1, 'Amit', 'Pune'), (2, 'Neha', 'Mumbai'), (3,
'Rahul', 'Nagpur');
INSERT INTO Product VALUES (101, 'Laptop', 55000), (102, 'Mobile', 20000),
(103, 'Headphones', 2500);
INSERT INTO Orders VALUES (1001, 1, 101, 1), (1002, 2, 102, 2), (1003, 1,
103, 3);

-- Joins
SELECT customer_name, product_name, quantity FROM Orders INNER JOIN Customer
ON Orders.customer_id = Customer.customer_id INNER JOIN Product ON
Orders.product_id = Product.product_id;
SELECT customer_name, order_id, product_name FROM Customer LEFT JOIN Orders
ON Customer.customer_id = Orders.customer_id LEFT JOIN Product ON
Orders.product_id = Product.product_id;
SELECT product_name, customer_name FROM Orders RIGHT JOIN Product ON
Orders.product_id = Product.product_id LEFT JOIN Customer ON
Orders.customer_id = Customer.customer_id;

-- Corrected FULL OUTER JOIN (MySQL Workaround using UNION)
SELECT C.customer_name, P.product_name
FROM Customer C
LEFT JOIN Orders O ON C.customer_id = O.customer_id
LEFT JOIN Product P ON O.product_id = P.product_id
UNION
SELECT C.customer_name, P.product_name
FROM Customer C
RIGHT JOIN Orders O ON C.customer_id = O.customer_id
RIGHT JOIN Product P ON O.product_id = P.product_id;

SELECT customer_name, product_name FROM Customer CROSS JOIN Product;
SELECT A.customer_name AS Customer1, B.customer_name AS Customer2, A.city
FROM Customer A JOIN Customer B ON A.city = B.city AND A.customer_id <>
B.customer_id;

-- Views
CREATE VIEW Order_Details_View AS SELECT order_id, customer_name,
product_name, quantity, price, (quantity * price) AS total_amount FROM Orders
JOIN Product ON Orders.product_id = Product.product_id JOIN Customer ON
Orders.customer_id = Customer.customer_id;
show tables;
SELECT * FROM Order_Details_View;
desc Order_Details_View;

CREATE VIEW Pune_Customers AS SELECT customer_name FROM Customer WHERE city =
'Pune';
show tables;

UPDATE Pune_Customers SET customer_name = 'Amit Kumar' WHERE customer_name =
'Amit';
select * from Pune_Customers;
```

```
DROP VIEW Order_Details_View;
show tables;


-- ========================================
-- ASSIGNMENT 6: Subqueries
-- ========================================

-- Subqueries
SELECT customer_name FROM Customer WHERE customer_id IN (SELECT customer_id
FROM Orders);
SELECT customer_name FROM Customer WHERE customer_id NOT IN (SELECT
customer_id FROM Orders);
SELECT product_name, price FROM Product WHERE price = (SELECT MAX(price) FROM
Product);
SELECT order_id, (SELECT price FROM Product WHERE Product.product_id =
Orders.product_id) * quantity AS total_amount FROM Orders;

SELECT customer_id, total FROM (
    SELECT Orders.customer_id, SUM(Product.price * Orders.quantity) AS total
    FROM Orders JOIN Product ON Orders.product_id = Product.product_id
    GROUP BY Orders.customer_id
) AS Purchase
WHERE total > 30000;

SELECT product_name, price FROM Product P WHERE price > (SELECT AVG(price)
FROM Product);

SELECT customer_name FROM Customer C WHERE EXISTS (SELECT 1 FROM Orders O
WHERE O.customer_id = C.customer_id);
```